



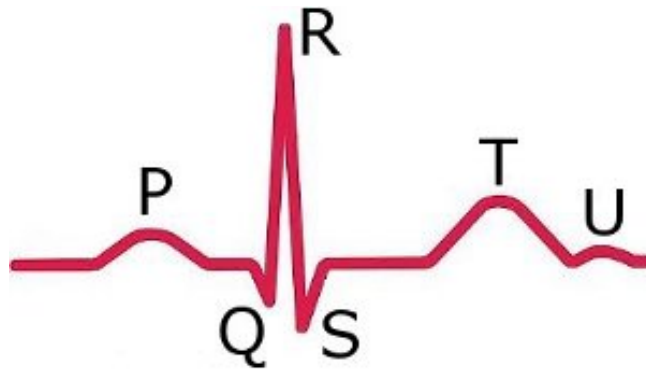
THE UNIVERSITY OF
MELBOURNE

FINAL REPORT

Respiratory Rate Estimation via ECG data

Author:
Luke Kuflik (1463828)

Subject Handbook code:
COMP-90072



Source: https://simulide.com/p/storage/2023/11/ecg_signal.jpg

Faculty of Science
The University of Melbourne

Original Version: 1st March 2018

Subject co-ordinators: A/Prof. Suzie Sheehy & Prof. Christian Reichardt

Contents

1	Introduction	2
1.1	Motivation and Biological Background	2
1.2	Research Methodology	3
2	ECG Raw Data	4
2.1	Data Processing	4
2.2	Preliminary Signal Exploration	5
3	Locating RR peaks	7
3.1	Filtering and Pre-Processing	7
3.2	Basic Algorithm	8
3.3	Algorithm Evaluation	8
3.4	Algorithm Extensions	9
4	Evaluation of Estimation Methods	11
4.1	Spectrum Analysis Methods	11
4.2	RSA vs RPA Performance	13
5	Conclusion	16
5.1	Result Discussion and Conclusion	16
5.2	Area for future research	16
	Bibliography	16

1 Introduction

This report is submitted as part of the assessment for the authors enrolment in Masters class The Art of Scientific Computation at the University of Melbourne. The report outlines the process the author went through when completing the Respiratory Rate research project.

1.1 Motivation and Biological Background

Respiratory rate, defined as the number of complete breaths per minute, is a fundamental vital sign, with importance across healthcare and exercise science [1],[7]. A persons Respiratory Rate is impacted by a number of factors including injury, illness or stress, adding difficulty to the measurement process, however it is an extremely important vital as abnormal Respiratory Rate is often the first indicator of illness [1]. Despite this, Respiratory Rate monitoring is still not performed routinely in the healthcare setting, which may be due to the lack of standard Respiratory Monitoring technology [7]. One area of critical importance in Respiratory Rate Estimation is finding a low-cost, consistent and robust method for measuring Respiratory Rate, this is most critical in rural, low income areas where the lack of accurate, easy to obtain measurements may led to patients being incorrectly sent home [13], [7]. This issue is pertinent to diagnosis of Pneumonia, especially in children; Pneumonia is responsible for 14% of all deaths of children under 5, despite pneumonia being easily treatable with widely available oral medication, indicating the importance of being able to effectively diagnose in settings where resources are scarce [17]. Respiratory Rate can be an indicator of severe pneumonia, and the World Health Organisation suggest that Respiratory Rate is integral to the diagnosis pathway; further illustrating the criticality of determining reliable methods for measurement. [7].

An area of active research is developing a method for reliably estimating Respiratory Rate via electrocardiogram (ECG) signals. ECGs are a non-invasive, widely available tool used in healthcare that measures the electrical activity of the heart, specifically volatage over time. Electrodes (typically 10) are placed on a person lying down, these electrodes measure the magnitude of the electric potential from 12 different angles - this is called a 12 lead ECG [16]. A normal ECG signal is made up of 3 parts, the P-wave, QRS complex and T wave associated to different activities of the hearts beating cycle [16]. Figure 1.1 illustrates the characteristics of each part.

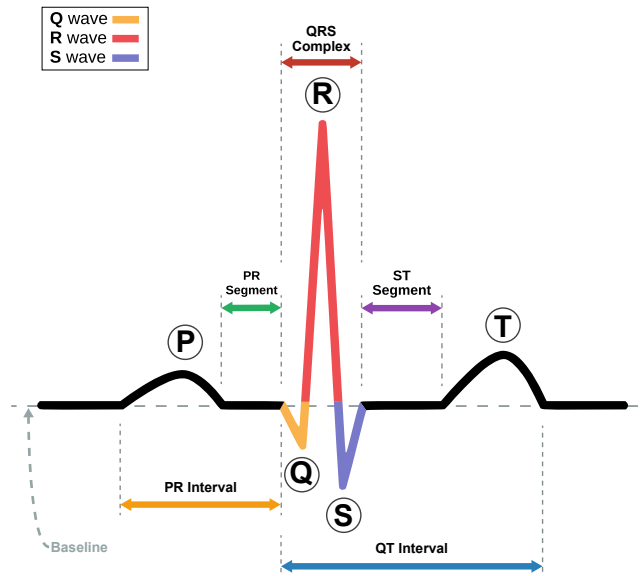


Figure 1.1: Notated diagram of a Sinus Rhythm, the signal generated by an ECG [16].

Of interest to this project is the QRS complex characterised by large short peaks on the ECG signal, it corresponds to repolarisation of the ventricles, that is the contraction of the heart to pump out blood [16]. There are two commonly used methods that use waveforms create by these peaks (sometimes refered to as RR peaks), to estimate Respiratory Rate providing a potential low-cost widely available methods for measuring Respiratory Rate. The first challenge will thus be, being able to robustly identify the location of QRS complexes in an ECG signal, which poses a challenge due to the sensitivity of ECG signals to noise and artifacts.

1.2 Research Methodology

The predominant Goal of this project is two fold, firstly we develop a method for identifying QRS complexes on an ECG signal, from which we use this algorithm to evaluate methods for estimating Respiratory Rate from different waveforms that stem from patterns in the QRS complexes occurrence and amplitude. The methods we will evaluate analyse characteristics of the waveform namely how the time between QRS complexes varies - referred to as the RSA method or how the amplitude of the waveform varies - referred to as the RPA method. We will employ spectrum associated methods (Fast Fourier Transform or Lomb-Scargle) to try and identify a dominant, roughly consistent frequency in the expected Respiratory Rate range that will become the estimated Respiratory Rate.

Our Methodology will go as follows:

1. Investigate ECG signals, identifying datasets with distinct varying behaviour to test the Robustness of our algorithms. This is described in Section 2.
2. Develop, evaluate and refine an algorithm to robustly identify RR peaks, testing the performance against annotations for the occurrence of a QRS complex. This is described in Section 3.
3. Evaluate and test different methods for estimating Respiratory Rate, by analysing their ability to detect a strong, consistent frequency that with high probability is the Respiratory Rate. This is described in Section 4.

2 ECG Raw Data

The data used in this project is sourced from the Apnea-ECG dataset [9], [11]. There are 75 datasets each corresponding to the sleep of an individual, they contain an ECG signal with amplitude in millivolts, along with annotation files. Every dataset has a QRS annotation file that contain time annotations for the occurrence of QRS Complexes, whilst 35 of the files contain a further annotation file indicating the presense of Apnea in minute increments [9]. The data is stored using WFDB conventions.

Figure 2.1 encapsulates the form we want to convert the data and annotations into, i.e an array of waveform values and an associated time array, along with arrays that capture annotation value and time of occurence.

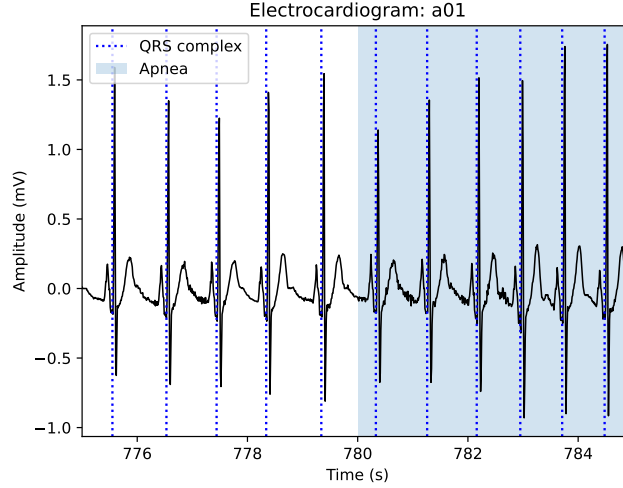


Figure 2.1: ECG signal along with QRS and Apnea Annotations.

2.1 Data Processing

To begin to analyse the ECG signal, we first need to understand how the data is stored. The data for the ECG waveform is stored in binary, and the way to parse the data is described by a header file that is written in ASCII. The description for how to read these header files is provided by [10], under WFDB file formats (header). A header file contains two lines, the record line contains information that is relevant to all signals, for us the important information is regarding sampling frequency and number of signals, allowing us to extract our timing information. The next step is to understand the signal, this is described by the signal line. The key details are the format for which to extract the signal (this specifies how to convert binary into an integer data type), the specific formats are described by [10], under WFDB file formats (signal). In the case of this project, the data is stored in format 16, that is in 16 bits stored least significant byte first (i.e little endian) [9], [10]. This means the integer is given by arranging the bytes as second byte followed by first byte. Additionally, there is information about the ADC (analog to digital conversion), of relevance to this project is the ADC gain, dividing by this value will scale the integer data into mV ready now for us to plot.

Extracting annotations requires a little more work, and the full description for how to go about this is given by [10], under WFDB file formats (annot). The overview is as follow, the data comes in a pair; an A value and an I value, the A value corresponds to the annotation in our case normal or Apnea occurrence and the I value whichs specifies how long after the previous annoation this annotation occured, for the first one that is time after the start for which it occured. The information for these two comes in 16 bits, the 6 most significant bits (in our case the last 6 as its implemented least significant byte first) correspond to the annotation, and the first 10 bits correspond to the interval. There is one caveat, that is if the annotation gives the numerical value 59 (note there are other values but these are irrlevant to this projects data), this specifies a skip and needs to be handled differently, as it means the next 4 bytes of information tell you an extended time interval for which to add to the interval of the upcoming annotation (the next annotation takes longer to occur). The binary information of the next 4 bytes can be thought of as two chunks of bytes, the first being the high and the last the low, each chunk read using the little endian method as we have done before. Then the interval is given by the integer associated with high chunk then low chunk.

Figure 2.2 summarises how each data form is read.

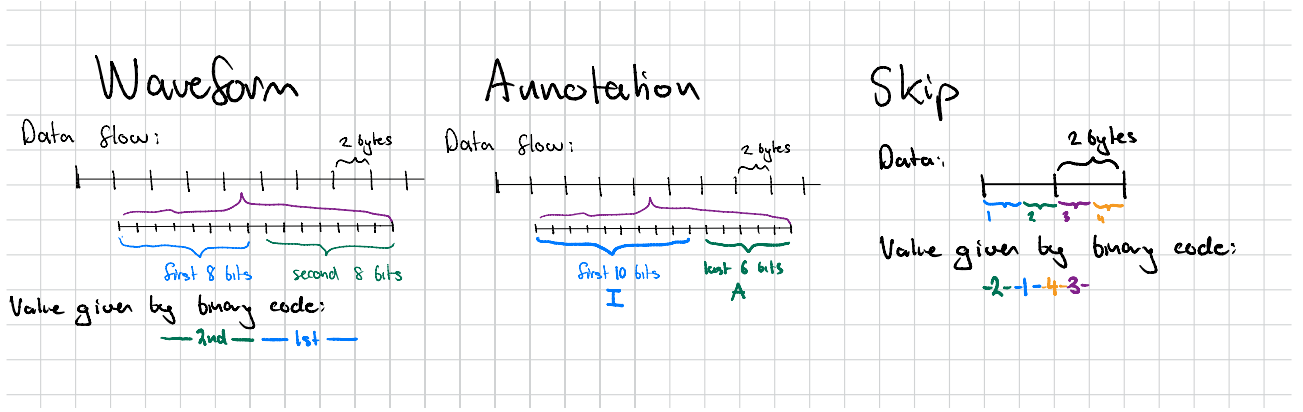


Figure 2.2: Visual for how to process each data form.

2.2 Preliminary Signal Exploration

As we are working with ECG signals, the amplitudes and time intervals differ across time and samples due to morphological differences in humans, along with noise and artifacts picked up in the data collection process [13]. We now illustrate a few samples that show these differences, these samples will give a basis of data to check our algorithms for robustness.

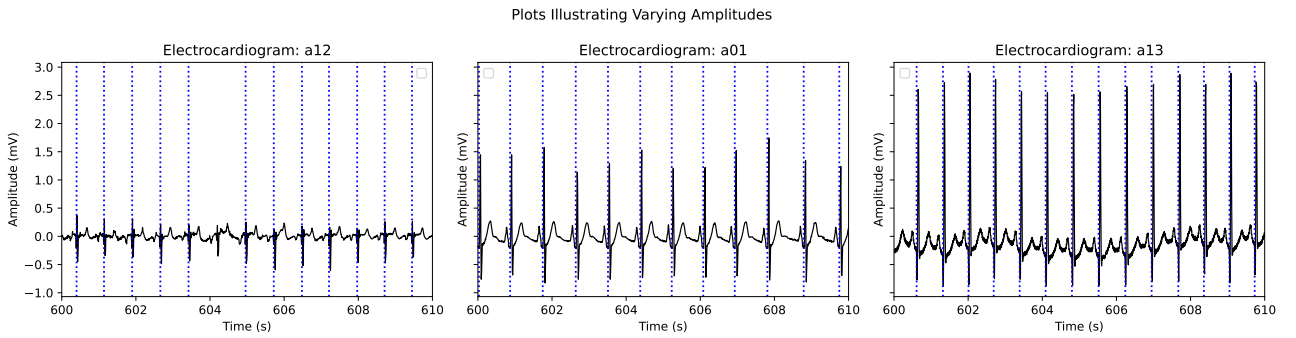


Figure 2.3: ECG datasets illustrating differing amplitude magnitudes.

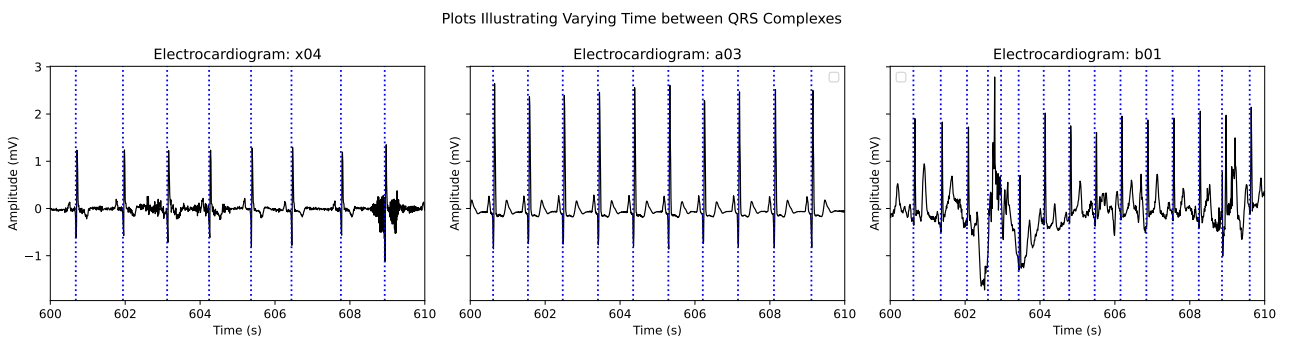


Figure 2.4: ECG datasets illustrating differing time intervals between QRS Complexes.

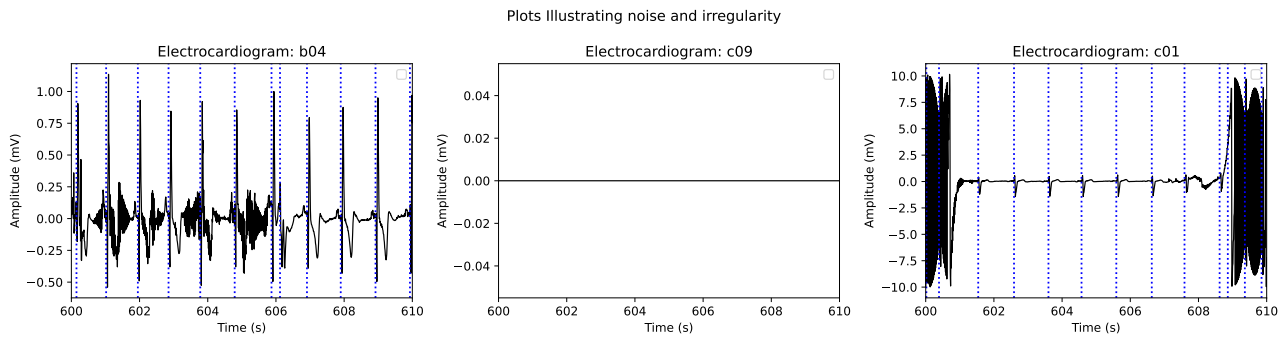


Figure 2.5: ECG datasets illustrating the existence of noise and artifacts in the data.

Figure 2.3 shows how the amplitude of the QRS signal can vary significantly across datasets, Figure 2.4 illustrates that the time between QRS complexes also varies, and Figure 2.5 highlights the existence of noise and artifacts in some datasets. Further, we can also see in the following (Figure 2.6) that within a data set the QRS complex amplitude can be only slightly larger than the amplitude of other peaks in the data.

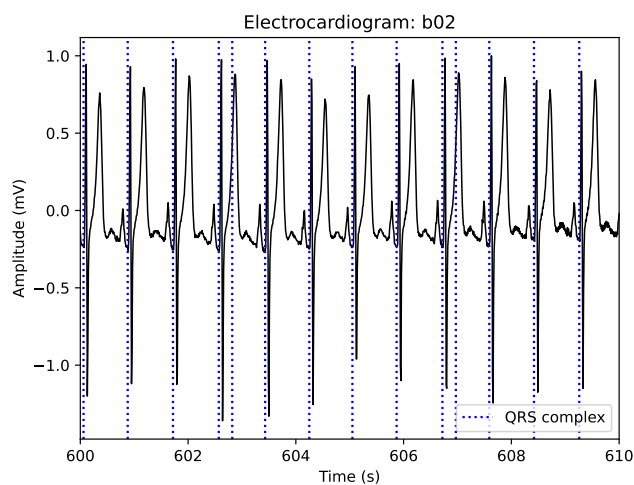


Figure 2.6: ECG signal illustrating QRS amplitude can be close to other peak amplitudes.

Going forward we focus on testing our algorithms against these data samples: a01, a12, a13, a03, x04, b01, b02, c02, as they represent how amplitude of QRS complexes, and the time between can vary. This is important as those characteristics will form the basis of our Respiratory Rate estimation and we need to ensure that the RR peak identification algorithm is robust against variation in these traits.

3 Locating RR peaks

In this section we develop and evaluate methods for identifying QRS complexes otherwise known as RR peaks in ECG data. The method we develop is inspired by the Pan-Tompkin Algorithm [8]. The Pan-Tompkins algorithm is widely used in literature for ECG analysis, [5], hence why we use it as inspiration in this project. We implement simplifications of the process whose performance we evaluate against the QRS annotations supplied.

3.1 Filtering and Pre-Processing

ECG signals are prone to noise, and variation in the data, so its important to filter and pre-process before attempting to locate RR peaks. The filtering is based off literature values for ECG signals, specifically we expect to be locating frequencies associated within the range of 5 - 15 Hz [8]. Figure 3.1 illustrates an example waveform before and after filtering in different frequency ranges, employing a Fourier Transform before removing frequencies and inverting back.

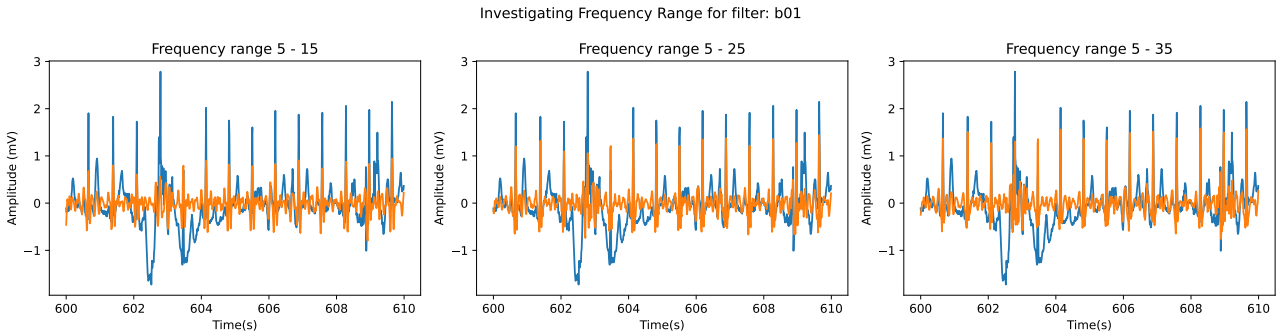


Figure 3.1: Frequency Filters applied in different ranges

We can see that a frequency filter cleans up the waveform to better resemble an ECG signal, whilst retaining the QRS complexes, Figure 3.1 suggests that a range of 5-35 Hz best maintains Amplitude of the QRS complex along with an ECG signal shape, this is the frequency range this project uses.

Once we restrict ourselves to a filtered waveform, we implement a Pre-Processing method heavily influenced by [8]. The idea is that RR peaks are associated with large inclines and declines in the wave form, so we want to use Derivatives, and square them to highlight and exhausterbate this difference, before a moving integration window converts the derivative back into a smooth signal with a large peak associated with the RR peaks whilst minimising the effect of the other smaller peaks. The process is specifically,

1. Differentiate the waveform.
2. Square the differentiated waveform, to ensure all values are positive, additionally the steeper incline and decline , associated with the peaks will now be amplified helping to distinguish against the smaller peaks.
3. A moving integration window on the Squared Differentiated waveform, produces a signal that amplifies the RR peak (nearby it), and minimises the other smaller peaks.

Figure 3.2 illustrates how the waveform is transformed over this process.

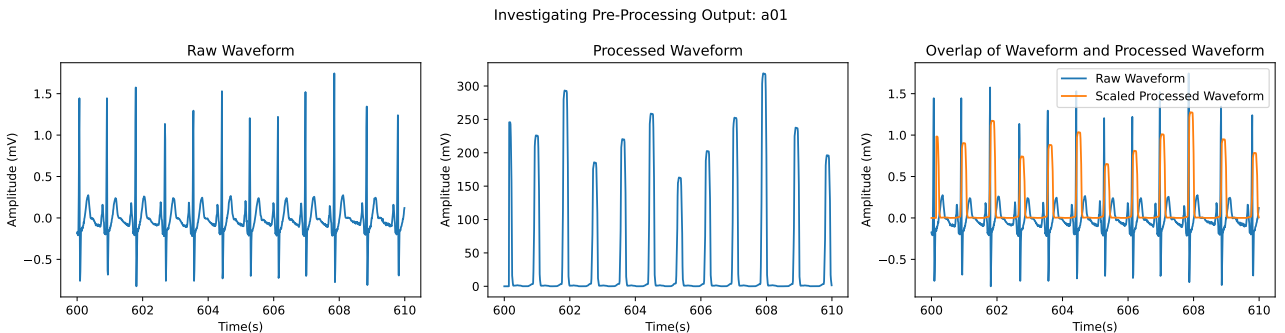


Figure 3.2: Illustration of the Pre-Processed Waveform

3.2 Basic Algorithm

We can see in Figure 3.2 that the processed waveform has large peaks close to (but just after) the location of the RR peaks in the Signal. The idea of our algorithm is to then locate the peaks on the processed waveform which are clear and can be done using an inbuilt python peaking finding function (`sp.signal.find_peaks`). We then in a small search window around the location of these peaks, move back onto the original waveform and in this interval identify the largest value and the time associated with it, this is what the algorithm identifies as a peak. The search window is chosen as 0.25 secs, this is not too large as the normal heart rate is from 60-100 bpm corresponding to 0.6 - 1sec between QRS time complexes [15]. When implementing the inbuilt function, we need to prescribe a threshold for what we expect a peak in the integrated waveform to be above. As we have seen that amplitudes can vary significantly, the idea of our algorithms is to then start with a high threshold value, we choose 1000 as the dataset with the largest amplitudes we identified (a13) was roughly 700 (note the scaling is 1/200), so 1000 was chosen as a buffer. Starting from the high threshold we see how many peaks are identified, if not enough peaks are identified with respect to some ratio of the time interval we gradually decrease the threshold until the number of peaks identified is above the minimum number we expect to identify. This is the basic idea of the algorithms developed in this section, Figure 3.3 provides a visual summary of the Pre-Processing and search window idea.

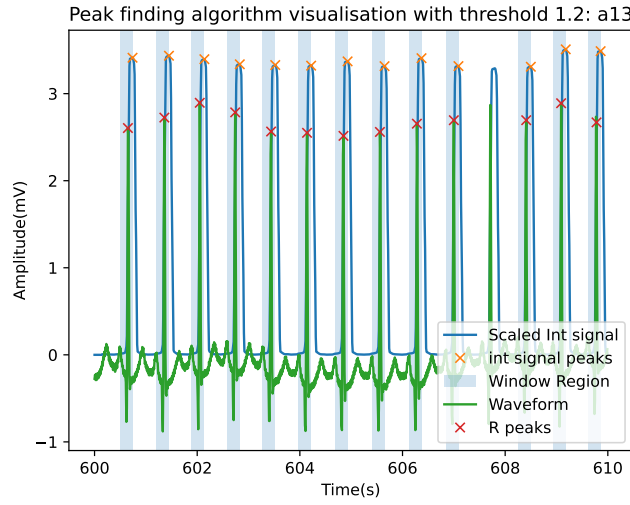


Figure 3.3: Plot illustrating the basic idea of the peak finding algorithms developed in this paper

The pseudocode below summarises the basis idea of the base algorithm.

Algorithm 1 Basic Algorithm form

```

filter_signal ← Filter(signal)
processed_signal ← Preprocessing(filter_signal)
threshold ← 1000
length_peaks ← 0
while length_peaks < (end.time – start.time)time_interval_threshold do
    peaks ← findpeaks(processed_signal, threshold)
    length_peaks ← length(peaks)
    threshold = threshold – 10
end while
for  $i = 1, \dots, \text{length}(\text{peaks})$  do
    region ← filter_signal[search_range( $i$ )]
    peaks_indices[ $i$ ] ← argmax(region)
end for

```

3.3 Algorithm Evaluation

In Figure 3.3 we can see that with a threshold of 1.2 peaks per second that peak has been missed. To test which threshold is most effective, we test the number of RR peaks each algorithm identifies against the number of QRS annotations, this is summarised by Table 3.1.

Table 3.1: Proportion of peaks detected for different threshold values between 0.8 and 1.4 in the first hour of data.

Threshold	0.8	0.9	1.0	1.1	1.2	1.3	1.4
a01	0.74	0.87	0.92	1.01	3.14	3.14	3.14
a12	0.72	0.88	0.88	0.98	3.04	3.04	3.04
a13	0.6	0.67	0.76	0.82	0.91	0.97	2.76
a03	0.83	0.92	0.99	3.51	3.51	3.51	3.51
x04	1.05	1.19	5.17	5.17	5.17	5.17	5.17
b01	0.75	0.86	0.96	1.0	3.16	3.16	3.16
b02	0.74	0.74	0.86	0.9	2.7	2.7	2.7
c02	0.89	0.96	1.06	3.85	3.85	3.85	3.85
Mean Detected	0.79	0.88	1.45	2.15	3.18	3.19	3.42

We can see that different time interval ratios are better than others, namely between 0.90 and 1.0, investigating between 0.90 to 1.0, we have:

Table 3.2: Proportion of peaks detected for different threshold values between 0.9 and 1.0 in the first hour of data.

Threshold	0.9	0.92	0.94	0.96	0.98	1.0
Mean Detected	0.88	1.39	1.41	1.44	1.44	1.45

So the optimal number of RR peaks per second threshold value seems to be 0.90, which on average correctly identifies 88% of peaks.

3.4 Algorithm Extensions

The problem with a fixed time interval ratio is the variability in the time between QRS complexes across datasets. The following extensions of the algorithm described above attempt to get around this problem in different ways.

One way we could is by noticing that a large jump often occurs from values that are close to the estimated correct number of peaks to a number far higher, intuitively this is because we are now factoring in other peaks (T, P for example). The first extension algorithm detects this jump, and then chooses the peaks identified from the time interval threshold before that jump. When detecting this jump we test time interval thresholds between 0.5 to 1.6 as the results indicate these capture all jumps we expect. Table 3.3 summarises the performance of this algorithm.

Table 3.3: Proportion of peaks detected for the jump based algorithm

a01	a12	a13	a03	x04	b01	b02	c02
1.01	0.98	0.97	0.99	1.19	1.0	0.9	1.06

Its clear from this table that this algorithm performs far better than any of the fixed time interval threshold based algorithms, on average it correctly identifies 101% of peaks, noting that it before well on most datasets, with notably worse performance for x04 and b02. The sacrifice with this algorithm is that the increased complexity reduces the efficiency of the algorithm. Table 3.4 illustrates that the run time over an hour of data increases up to about 15 times.

Table 3.4: Time Comparison (sec) for running each algorithm over an hour of data

	a01	a12	a13	a03	x04	b01	b02	c02
Jump Algo	1.61	2.27	0.81	2.0	2.31	1.75	1.87	1.96
Basic Algo with Threshold 0.9	0.18	0.27	0.12	0.16	0.28	0.2	0.22	0.19

In the basic algorithm the while loop gets replaced with the following, its clear how the time complexity increases.

Algorithm 2 Jump Based Modification to location of Processed Signals Peaks

```

threshold  $\leftarrow$  1000
initialise arrays to store peaks (peaks_list) and number of peaks identified (peaks_size)
for  $i = 5, \dots, 16$  do
    time_interval_threshold  $\leftarrow i \times 0.1$ 
    length_peaks  $\leftarrow 0$ 
    current_threshold  $\leftarrow$  threshold
    while length_peaks < (end_time - start_time)time_interval_threshold do
        peaks  $\leftarrow$  find_peaks(processed_signal, threshold)
        length_peaks  $\leftarrow$  length(peaks)
        current_threshold = current_threshold - 10
    end while
    peaks_list[ $i - 5$ ]  $\leftarrow$  peaks
    peaks_size[ $i - 5$ ]  $\leftarrow$  length(peaks)
end for
found  $\leftarrow$  False
for  $j = 1, \dots, \text{length}(\text{peaks\_size}) - 1$  do
    if peaks_size[ $j + 1$ ] / peaks_size[ $j$ ]  $\geq 2$  then
        peaks = peaks_list[ $j$ ]
        found  $\leftarrow$  True
        break
    end if
end for
if found = False then peaks = peaks_list[-1]
end if

```

Another improvement that is only relevant to the data sets used in this project, is by basing the time interval threshold as a factor (something around 0.9-0.95) of the number of QRS annotations, this is clearly not a very viable algorithm however has it can't be used on other datasets but is worthwhile evaluating as is if its robust on these datasets it may be a good candidate when we test Respiratory Rate estimation methods. Tables 3.5, 3.6 summarise the performance of this algorithm.

Table 3.5: Proportion of peaks detected for the annotation based algorithm with factor 0.90

a01	a12	a13	a03	x04	b01	b02	c02
0.92	0.98	0.91	0.92	0.9	0.96	0.91	0.94

Table 3.6: Proportion of peaks detected for the annotation based algorithm with factor 0.95

a01	a12	a13	a03	x04	b01	b02	c02
0.97	0.98	0.96	0.96	0.96	0.96	2.7	0.96

With a factor of 0.90 the average perctange of peaks detected is 93%, and for a factor of 0.95 it is 118% (due to the heavy influence of b02). Here the algorithm is almost identical to the basic algorithm form with the modification that:

$$\text{time interval threshold} = \text{factor} \times \frac{\text{number of annotations}}{\text{end time} - \text{start time}}$$

Of these two possible extensions of the algorithms its clear that the jump detection based algorithm is the best performing, with a sacrifice on efficiency - which gets amplified in the Respiratory Rate estimation process.

4 Evaluation of Estimation Methods

When estimating Respiratory Rate from ECG signal, the common approach is to isolate RR peaks, and then work with the waveform associated with amplitude and time of the RR peaks [6]. There are two common methods; the first focuses on locating the dominate frequency in the time between RR peaks, this corresponds to the biological phenomenon of Respiratory Sinus Arrhythmia (RSA), which states that heart rate speeds up when you breath in and slows down when you breath out [4], [6]. The data we are working with here is 2D, with each pair corresponding to the length of time between RR peaks and the midpoint of that time interval. The other method is based on the amplitude differences of the RR peaks (RPA), which is associated to impacts breathing in and out has on the electrical output of the heart [6]. The data we are working with here is similarly 2D with each pair corresponding to the amplitude of Peak and the time at which it occurred. The idea behind both methods is to then isolate the frequency with the strongest associated amplitude, where the assumption is that this frequency corresponds to the Respiratory Rate whilst the less dominate frequencies correspond to noise and other biological processes and artifacts. The normal Respirate Rate range is 12 - 20 BPM [3], which corresponds to a range of 0.2- 0.33Hz , for a candidate Respiratory Rate. Note that we can't ignore frequencies outside of the range especially as we know we are dealing with data where Apnea and possible other Respiratory events are occurring that cause Respiratory Rate to land outside of the normal range - hence we a predominantly looking for a consistent dominate frequency within a range of 0.05 - 0.50 Hz (to avoid noise). In all of the work below, the RR peaks are identified using the Jump Detection Algorithm described above.

4.1 Spectrum Analysis Methods

Implementing either of these methods requires spectrum analysis, however as we are dealing with non-uniform time data we can't blindly apply the Fast Fourier Transform (FFT). There are two ways of dealing with non-uniform time series data which we explore in this project. The first is to use an interpolation method between the RR peaks and then sample a uniform time from this and apply FFT to this waveform, the other is to use the Lomb-Scargle Method defined for non-uniform time series data. We give a brief explanation of each method, and compare the output spectrum of those in this section, then the section below we will continue the analysis to the spectrograms.

4.1 Interpolation and FFT method

The idea behind this method is to create an interpolated waveform, specifically by fitting a piecewise cubic function to the data and then sampling uniformly from this wave, this then means we have uniform time and are able to apply a discrete Fast Fourier Transform algorithm. As the time intervals doesn't vary largely, as breathing is typical consistent, there is validity to this method, in Figure 4.1, 4.2 its clear that the interpolation matches the waveform closely.

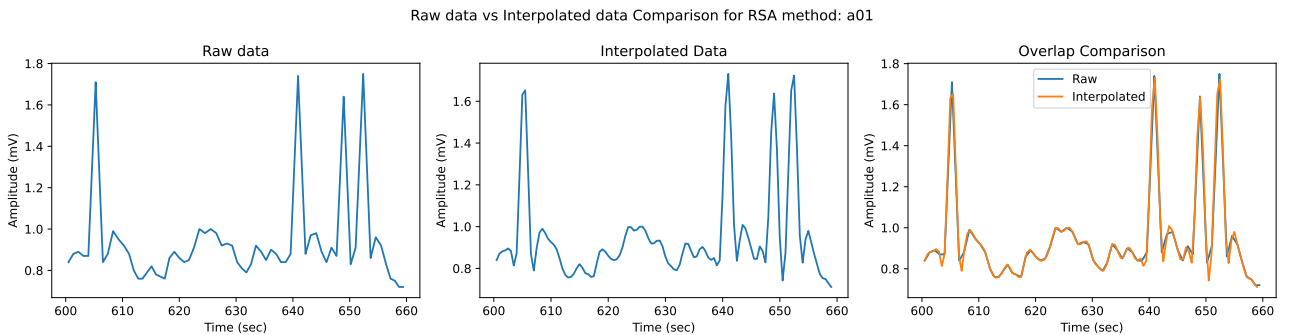


Figure 4.1: Raw data vs Interpolation comparison for RSA method

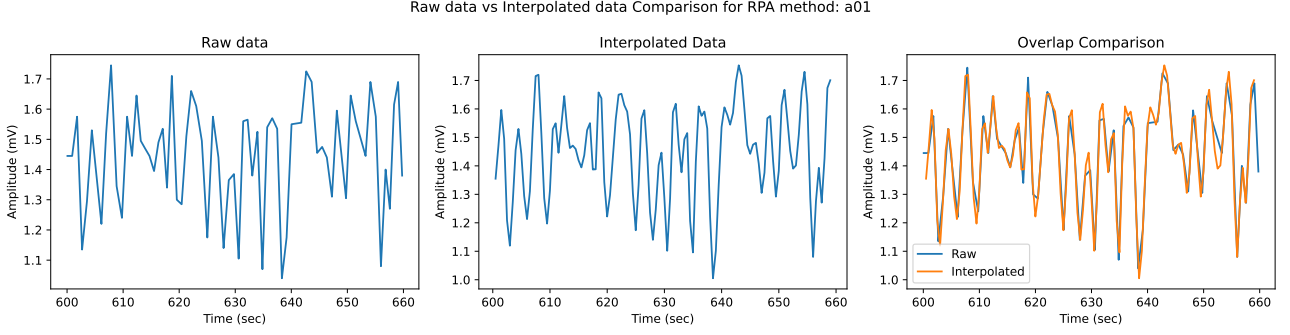


Figure 4.2: Raw data vs Interpolation comparison for RPA method

The discrete Fourier Transform (FFT) is a method that decomposes a sequence of points into its frequency components. Per the Nquist Theorem, we consider frequencies that are multiples of $1/\text{sample rate}$. For these frequencies, then when we consider a sample $(x_n), 1 \leq n \leq N$, where we have N samples. The Discrete Fourier Transform for frequency index k , is given by:

$$X_k = \sum_{n=1}^N x_n \cdot e^{\frac{-2\pi i k n}{N}}.$$

Note that X_k is often a complex number, where the magnitude corresponds to the amplitude, and the angle the phase. When plotting spectrums we are looking at the $|X_k|$ against its frequency k . When we refer to a Fast Fourier Transform this refers to an algorithm that computes the transform in the fastest known time complexity ($O(N \log N)$).

4.1 LombScargle Method

The reference, [14], provides a full description and discussion on the LombScargle Method, the method we implement is per SciPy, whose documentation is given by [12], the method SciPy implements is described by [18]. The basic idea is that the LombScargle Method is statistical, where each frequency θ is fitted to a function of the form,

$$f_{\theta}(x) = a * \cos(\theta x) + b * \sin(\theta x) + c.$$

The fitting is done via weighted least squares estimates. This method has no reliance on uniform time, and thus allows us to directly analyse the spectrum of non-uniform data without relying on first interpolating. Something worth considering when implementing this method is that as it is statistical thus there is a presence of error in the estimation.

4.1 Method Comparison

In Figures 4.3, 4.4 we can see a comparison of both spectrum analysis methods being applied to each of the RSA and RPA waveforms. The overlap plots show that clearly both analysis methods pick up on the same frequencies, however they do so with different amplitudes, in the RPA case the LombScargle Method picks up on large frequencies, whilst in the RSA case LombScargle and FFT methods are similar, with the FFT method amplitudes being slightly larger. These Figures indicate that both methods are viable, as they pick up similar frequency peaks with similar amplitude proportions (i.e amplitudes of frequency in relation to others within the same method). As a preliminary spectrum analysis doesn't conclusively point towards one of the spectrum methods, we evaluate the Respiratory Rate Method employing both methods.

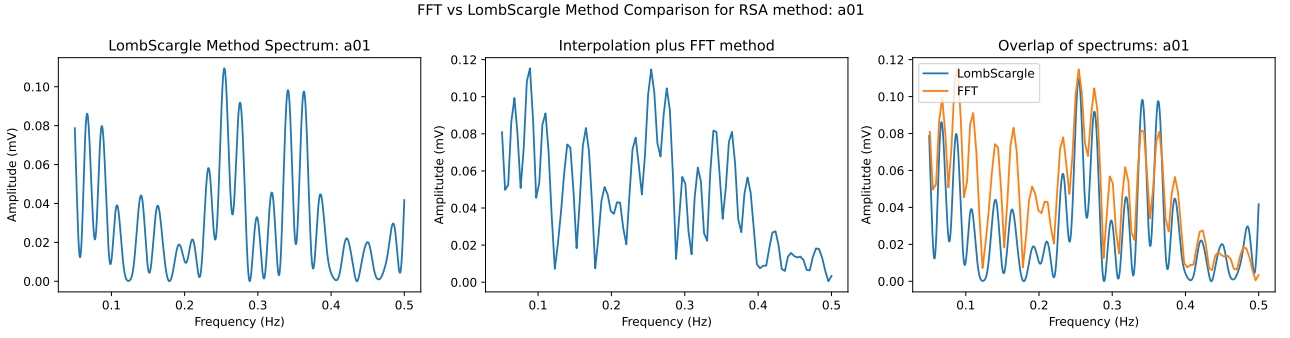


Figure 4.3: Spectrum comparison of LombScargle vs FFT based methods for RSA

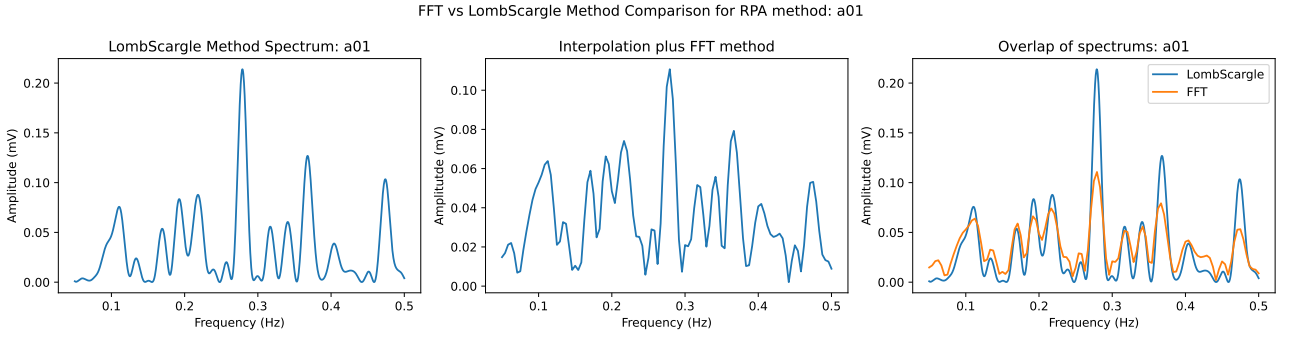


Figure 4.4: Spectrum comparison of LombScargle vs FFT based methods for RPA

In the following section we are going to produce Spectrograms, in producing these plots the time complexity of both methods becomes important, the Fast Fourier Implementation is $O(N \log N)$, whilst the LombScargle method is $O(NM)$, where M is the number of frequencies, N the number of data points. This difference becomes significant, and is why for compilation reasoning we focus largely on the FFT based method, which we shown to be valid.

4.2 RSA vs RPA Performance

We begin evaluating RSA vs RPA methods by looking at spectrums, in Figure 4.5, we can see in the overlap plots that both RSA and RPA methods, employing either the FFT or LombScargle spectrum analysis pick up on similar frequencies. However the a key thing to notice is in our expected Respiratory Rate interval (0.2 -0.33), RSA picks up on two frequency peaks whilst RPA only picks up on one, this is an indication that the RPA method may be able to isolate a dominate frequency better.

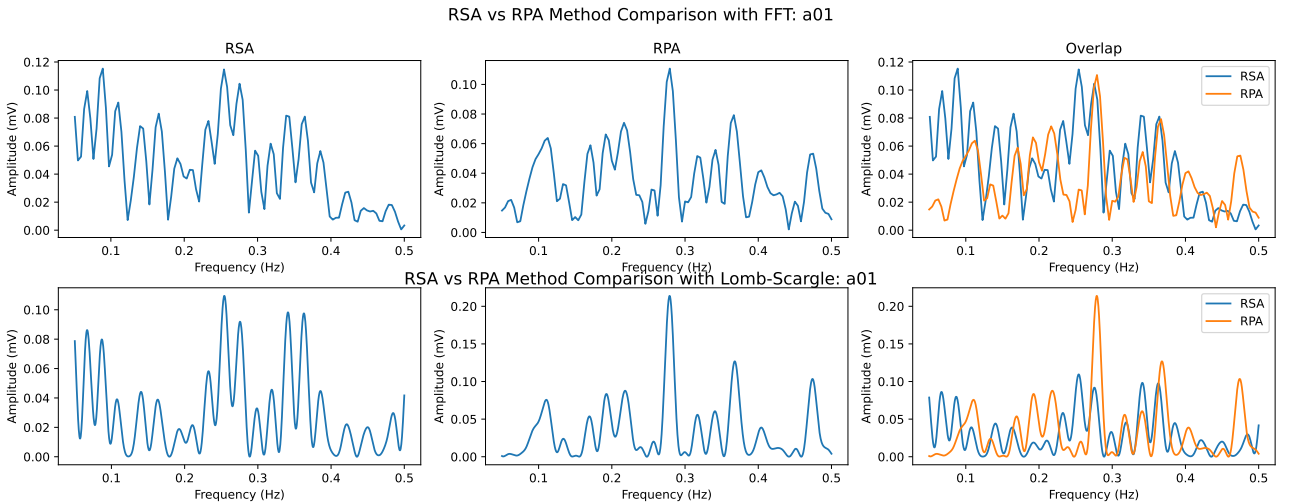


Figure 4.5: Spectrum comparison of RSA vs RPA method

To better evaluate each methods we now focus on analysing Spectrograms, which tell us how the spectrum changes over time, as we expect Respiratory Rate to remain relatively stable [2], we use spectrograms to see if we can pick up on a consistent dominate frequency within the expected range. If we are able to, that would be a good indication of the methods ability to estimate the Respiratory Rate. In what follows the spectrograms work on a 60sec window, with 10 sec increments, i.e produces a spectrum every 10secs, based on a 60sec window of data (30 secs each side). Figures 4.6, 4.7 we can see a Spectrogram over the first hour for each method, implemented with each analysis technique. Its clear in both figures that the RPA method picks up on a stronger dominate frequency sitting just below 0.3HZ compared to the RSA method, which picks up a similiar frequency however with a weaker amplitude.

Spectrogram comparison of RPA vs RSA methods implementing FFT: a01

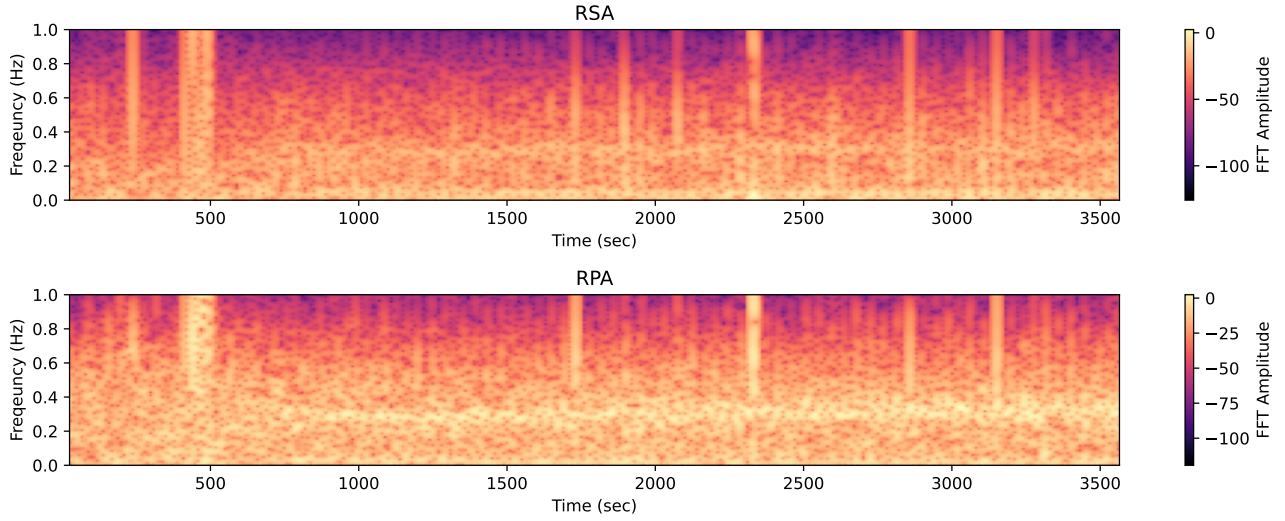


Figure 4.6: Spectrogram comparison of RSA vs RPA method with FFT implementation

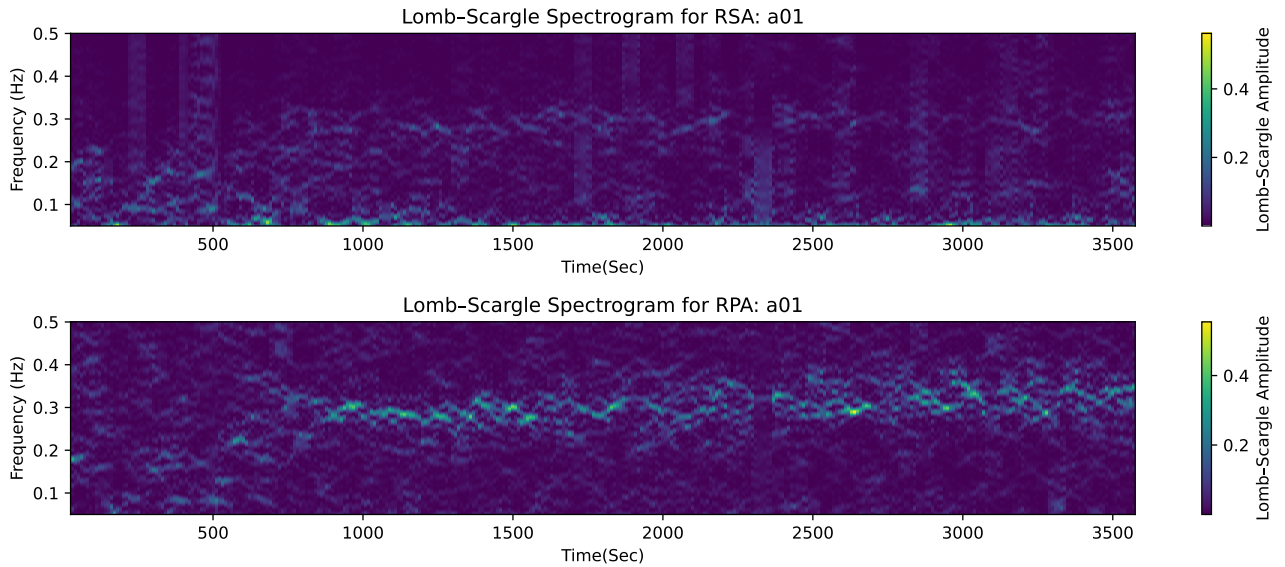


Figure 4.7: Spectrogram comparison of RSA vs RPA method with LombScargle implementation

Thus far it appears that the RPA method seems to perform better than the RSA based method. We now explore the performance of the RPA method, with the FFT implementation as that had the clearest consistent dominate frequency in the Spectrograms above, whilst also being far less computationally expensive. In Figure 4.8 we can see the Spectrogram of the RPA method applied to our 8 test datasets. In the Figure its clear the method lacks robustness as it is only able to detect a dominate frequency pattern in the a01 data set, in the other 7 no clear pattern is visually distinguishable. This is indicative of the noise that is present, indicating that the RPA method needs to be made robust to noise, this makes sense as the method is based upon amplitudes, which would be highly sensitive to artifacts and noise affecting the ability of the method to

pick up on the frequency associated with Respiratory Rate.

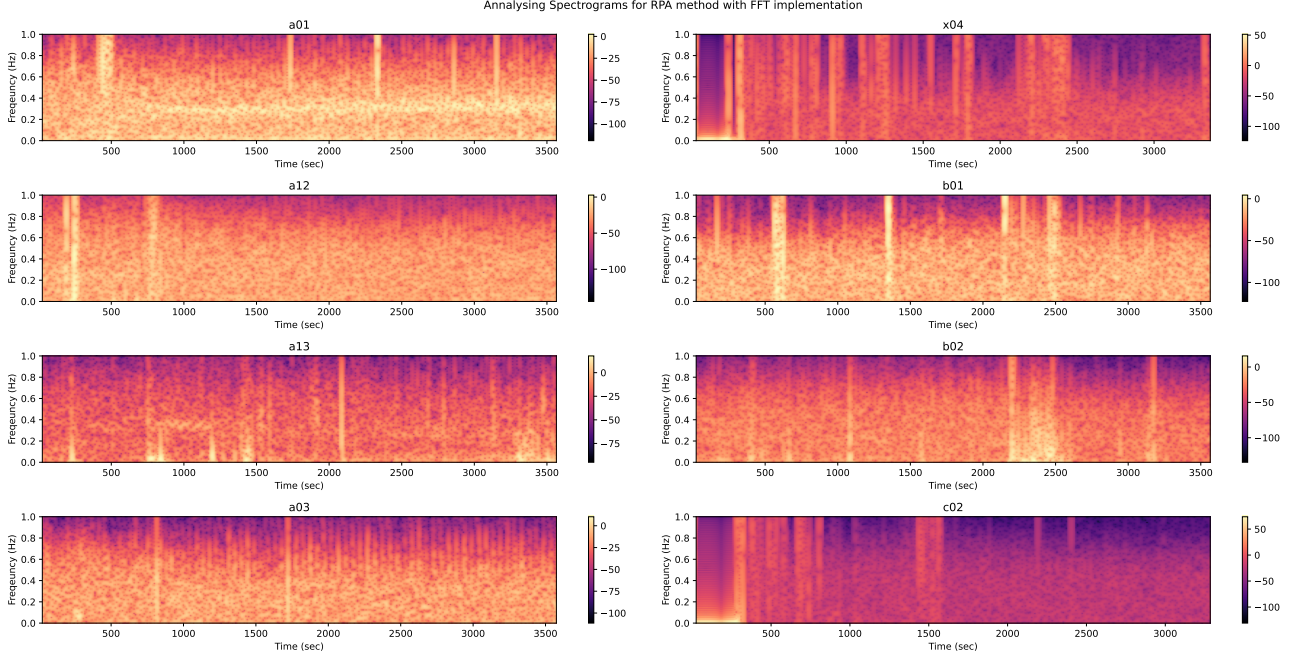


Figure 4.8: Spectrogram comparison across datasets for RPA method with FFT implementation

It is possible that the occurrence of QRS complexes is less sensitive to noise, so despite performing worse in the test case we now explore its performance, with FFT implementation, as shown in Figure 4.9.

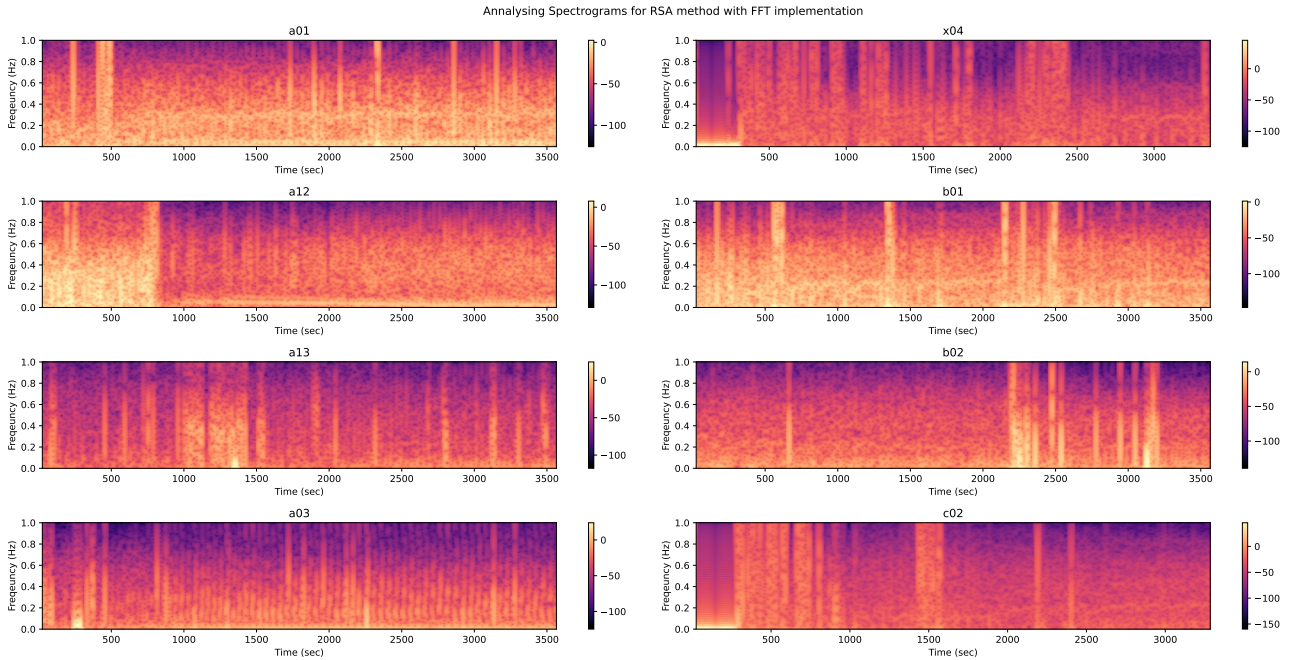


Figure 4.9: Spectrogram comparison across datasets for RSA method with FFT implementation

Looking at datasets b01, c02, b02 there are faint signs of a consistent frequency being picked up, most notably in dataset b01. This faint signal wasn't picked up by the RPA method, this indicates that whilst the RPA method may be better at picking up a stronger signal, it isn't as robust as the RSA method may be. In the Figures above we have only compared the FFT based implementations, the reasoning is that as discussed the LombScargle method is more computationally expensive, as so producing and saving the Spectrograms is time and memory consuming, further the LombScargle method being statistical introduces error into the estimation. We saw that the interpolation method is very similar in behaviour to the raw data providing validity for working with this method.

5 Conclusion

5.1 Result Discussion and Conclusion

5.1 Peak Detection Algorithm

We developed a basic algorithm model for identifying QRS complexes based upon a Processing Phase, and then Identification Phase using a adaptive threshold, we tested the performance of this algorithm against the QRS annotation files, noting that different time interval thresholds worked for different datasets. We then produced extensions of this algorithm to improve the performance, concluding with a Jump Detection Based Algorithm that produces the best performance, at the sacrifice of time complexity.

5.1 Respiratory Rate Methods

We evaluated the RSA and RPA Respiratory Rate estimation methods, using spectrograms as the predominant tool with the goal of identifying a dominate frequency, that persitted across time. We began by evaluating two methods for spectrum analysis of the non-uniform time series, a Discrete Fourier Transform with interpolation and LombScargle. We saw that the interpolation was very accurate, and due to the error involved in the LombScargle method along with time complexity, that the FFT based method was better for producing Spectrograms. We saw that the RPA method produced a stronger dominate frequency when it was able to detect one, however the RSA method was more robust across the datasets, however neither performed very well at all, with 7 of the 8 tested datasets not having a visually obvious pattern in the Spectrogram.

5.2 Area for future research

We suggeset the following directions for extending the work of this Project.

Peak Detection Algorithm:

- Implement statistical methods to determine false detection or missed detection of RR peaks, by identifying small or large gaps in RR peak intervals based off the average interval size. When detected remove, or adjust threshold.

Respiratory Rate Methods:

- Adjust Spectrogram increments and window sizes to see if this improves ability to detect signals.
- Estimate Error of the interpolation method to provide further validity of this FFT based method.
- Testing Methods with other known QRS complex algorithms, e.g Pan Tompkins.
- Implement unsupervised learning techniques to identify patterns in the Spectrogram that may correspond to Respiratory Rate that aren't visually apparent.

Bibliography

- [1] American Lung Association Editorial Staff. Understanding vital signs: The importance of your respiratory rate. URL: <https://www.lung.org/blog/respiratory-rate-vital-signs>.
- [2] S. R. Braun. Respiratory rate and pattern. In H. K. Walker, W. D. Hall, and J. W. Hurst, editors, *Clinical Methods: The History, Physical, and Laboratory Examinations*, chapter 43. Butterworths, Boston, 3 edition, 1990. URL: <https://www.ncbi.nlm.nih.gov/books/NBK365/>.
- [3] C. Chourpiliadis and A. Bhardwaj. Physiology, respiratory rate, 2022. Updated September 12, 2022. In: StatPearls [Internet]. Treasure Island (FL): StatPearls Publishing. URL: <https://www.ncbi.nlm.nih.gov/books/NBK537306/>.
- [4] Cleveland Clinic. Sinus arrhythmia: Causes, symptoms and treatment. URL: <https://my.clevelandclinic.org/health/diseases/21666-sinus-arrhythmia>.
- [5] M. A. Z. Fariha, R. Ikeura, S. Hayakawa, and S. Tsutsumi. Analysis of pan-tompkins algorithm performance with noisy ecg signals. *Journal of Physics: Conference Series*, 1532(1):012022, 2020. doi:10.1088/1742-6596/1532/1/012022.
- [6] L. Mirmohamadsadeghi and J.-M. Vesin. Respiratory rate estimation from the ecg using an instantaneous frequency tracking algorithm. *Biomedical Signal Processing and Control*, 14:66–72, 2014. URL: <https://www.sciencedirect.com/science/article/pii/S1746809414001001>, doi:10.1016/j.bspc.2014.07.003.
- [7] A. Nicolò, C. Massaroni, E. Schena, and M. Sacchetti. The importance of respiratory rate monitoring: From healthcare to sport and exercise. *Sensors*, 20(21):6396, Nov. 2020. doi:10.3390/s20216396.
- [8] J. Pan and W. J. Tompkins. A real-time qrs detection algorithm. *IEEE Transactions on Biomedical Engineering*, 32(3):230–236, 1985. doi:10.1109/TBME.1985.325532.
- [9] T. Penzel et al. The apnea-ecg database. In *Computers in Cardiology*, volume 27, pages 255–258, 2000. URL: <https://www.physionet.org/content/apnea-ecg/1.0.0/>, doi:10.1109/CIC.2000.898505.
- [10] PhysioNet. Wfdb applications guide. URL: <https://www.physionet.org/physiotools/wag/>.
- [11] T. Pollard, B. E. Moody, L.-w. H. Lehman, B. J. Gow, C. Fernandes, C. Xie, A. Johnson, R. G. Mark, and T. Heldt. PhysioNet as a global platform for biomedical research. *Nature Health*, 1(8):792–795, Aug. 2026. doi:10.1038/s44360-026-00096-z.
- [12] SciPy Developers. `scipy.signal.lombscargle`. <https://docs.scipy.org/doc/scipy-1.18.0/reference/generated/scipy.signal.lombscargle.html>, 2026. SciPy v1.18.0 documentation.
- [13] J. Valerian. The art of scientific computation: Respiratory rates. Technical report, Aug. 2026.
- [14] J. T. VanderPlas. Understanding the lomb–scargle periodogram. *The Astrophysical Journal Supplement Series*, 236(1):16, May 2018. doi:10.3847/1538-4365/aab766.
- [15] Victor Chang Cardiac Research Institute. What your resting heart rate says about you, 2026. URL: <https://www.victorchang.edu.au/resting-heart-rate>.
- [16] Wikipedia contributors. Electrocardiography. URL: <https://en.wikipedia.org/wiki/Electrocardiography>.
- [17] World Health Organization. Pneumonia in children. URL: <https://www.who.int/news-room/fact-sheets/detail/pneumonia>.
- [18] M. Zechmeister and M. Kürster. The generalised lomb–scargle periodogram: A new formalism for the floating-mean and keplerian periodograms. *Astronomy & Astrophysics*, 496(2):577–584, Mar. 2009. doi:10.1051/0004-6361:200811296.